

Esercizi giornalieri di Algoritmi
Percorso operativo fino all'esame

Dal 19 agosto al 10 settembre 2026

Da usare insieme a guida_algoritmi_30_lode.pdf

Regola: prima svolgere gli esercizi senza guardare le soluzioni. Nei feriali dedicare 60 minuti: 10 ripasso, 40 esercizi, 10 correzione. Nel weekend usare quattro blocchi: teoria, esercizi, Python, correzione. Ogni esercizio deve avere idea, pseudocodice, correttezza e complessità, anche se non si completa il codice.

Settimana 1: grafi fondamentali

Ripasso: liste di adiacenza, n , m , $O(n+m)$, grafo orientato e non orientato. **Esercizi obbligatori:** (1) costruire una lista di adiacenza da un elenco di archi; (2) contare gli archi senza doppio conteggio; (3) determinare la complessità di cinque frammenti con cicli; (4) verificare se un arco appartiene al grafo. **Python:** scrivere le funzioni e testarle su $n=0,1,5$. **Extra 30 e lode:** dimostrare la differenza di spazio tra lista e matrice.

Ripasso: marcatura, ricorsione, foresta DFS, costo $O(n+m)$. **Esercizi:** (5) implementare DFS ricorsiva; (6) elencare i nodi raggiungibili da s ; (7) contare le componenti connesse; (8) produrre padre, tempo di ingresso e tempo di uscita. **Extra:** mostrare l'invariante che garantisce che ogni nodo visitato e raggiunto da s tramite un cammino.

Esercizi: (9) verificare se un grafo è connesso; (10) verificare se è un albero; (11) trovare un ciclo in un grafo non orientato; (12) trovare i ponti con low e tin. **Extra:** dimostrare che un grafo con n nodi e $n-1$ archi è connesso se e solo se è un albero.

Blocco 1: rifare (6),(10),(12) senza appunti. **Blocco 2:** (13) classificare gli archi DFS; (14) rilevare un ciclo diretto con colori; (15) verificare se un grafo è un DAG; (16) trovare i punti di articolazione. **Python:** implementare colori e low-link. **Extra:** spiegare il caso speciale della radice DFS.

Blocco 1: (17) BFS da una sorgente; (18) ricostruire un cammino minimo; (19) trovare nodi a distanza k ; (20) trovare il diametro di un albero. **Blocco 2:** (21) ordinamento topologico con Kahn; (22) ordinamento con DFS; (23) provare che ogni DAG ha un nodo di grado entrante zero. **Extra:** nodi equidistanti da s,t .

Settimana 2: cammini pesati e greedy

Ripasso: rilassamento e pesi non negativi. **Esercizi:** (24) eseguire Dijkstra su un grafo di sei nodi mostrando la tabella; (25) implementare con heapq; (26) ricostruire il cammino; (27) calcolare distanza tra due insiemi. **Extra:** costruire e spiegare un controesempio con peso negativo.

Esercizi: (28) applicare Bellman-Ford per $n-1$ passate; (29) rilevare un ciclo negativo; (30) compilare una tabella Floyd-Warshall; (31) confrontare i tre algoritmi su quattro grafi. **Extra:** spiegare perché Floyd-Warshall usa la proprietà degli intermedi.

Esercizi: (32) calcolare un MST a mano; (33) implementare Union-Find; (34) implementare Kruskal; (35) dimostrare che Kruskal non crea cicli. **Extra:** usare la proprietà del taglio per giustificare ogni arco scelto.

Esercizi: (36) applicare Prim; (37) confrontare Prim e Kruskal su grafo denso; (38) trovare una strategia greedy per intervalli; (39) costruire un controesempio a una strategia greedy sbagliata. **Extra:** formulare scelta sicura e sottostruttura ottima per un problema a scelta.

Esercizi: (40) distinguere soluzione ammissibile, ottima ed euristica; (41) Vertex Cover da matching; (42) dimostrare fattore 2; (43) calcolare il rapporto su tre istanze; (44) spiegare perché una euristica non è una garanzia. **Extra:** scrivere una dimostrazione completa in forma d'esame.

Prima ora: rifare a memoria BFS, Dijkstra e Kruskal. **Seconda:** prova mista (45) SCC con Kosaraju; (46) condensato; (47) insieme minimo di sorgenti in un DAG; (48) minimo numero di archi per rendere un grafo fortemente connesso. **Terza:** codice. **Quarta:** correzione. **Extra:** spiegare tutte le complessità senza appunti.

Esercizi: (49) ricerca binaria; (50) risolvere cinque ricorrenze; (51) massimo sottosegmento; (52) k-esimo elemento; (53) coppia di punti più vicini. Extra: scrivere per uno di essi divisione, combinazione, correttezza e $T(n)$.

Esercizi: (54) Fibonacci top-down e bottom-up; (55) piastrellare $2 \times n$; (56) definire gli stati per stringhe ternarie senza pattern vietati; (57) contare le stringhe con una macchina degli stati. Extra: ridurre lo spazio da $O(n)$ a $O(1)$ dove possibile.

Esercizi: (58) massimo costo in matrice; (59) ricostruire il cammino; (60) esistenza di cammino in matrice binaria; (61) contare i cammini. Extra: gestire valori negativi e celle bloccate.

Esercizi: (62) zaino 0/1 con tabella; (63) ricostruire gli oggetti; (64) versione con spazio $O(C)$; (65) confrontare DP e backtracking. Extra: spiegare perché la ricorrenza non vale per lo zaino frazionario.

Esercizi: (66) tabella LCS; (67) ricostruire una LCS; (68) minima supersequenza; (69) verificare la relazione tra LCS e supersequenza. Extra: dimostrare la ricorrenza distinguendo caratteri uguali e diversi.

Esercizi: (70) massima sottosequenza palindroma; (71) riempire la tabella per diagonalmente; (72) sottosequenza crescente $O(n^2)$; (73) massimo sottosegmento $O(n)$; (74) numeri di Catalano. Extra: scrivere per ogni esercizio stato, base, ordine e complessità prima del codice.

Settimana 4: backtracking e simulazioni

Esercizi: (75) stringhe binarie; (76) palindroma; (77) stringhe bilanciate; (78) permutazioni; (79) n-regine. Per ogni algoritmo indicare configurazione, scelte, vincolo, potatura e caso base. Extra: dimostrare $O(nS(n))$ includendo il costo della stampa.

Svolgere una simulazione di tre ore usando una prova del corso: (80) ciclo Hamiltoniano; (81) zaino con backtracking; (82) matrici binarie con vincoli; (83) domanda teorica su correttezza. Quarta ora: correggere, classificare ogni errore e riscrivere l'esercizio più debole.

Rifare senza guardare (24),(34),(62),(70),(79). Per ciascuno scrivere soltanto prima lo stato o invariante, poi il codice. Preparare una pagina di errori: indici, casi base, complessità, rappresentazione e condizioni di applicabilità.

Svolgere in 60 minuti: (84) una DFS con ponte o articolazione; (85) BFS con cammino; (86) Dijkstra; (87) MST; (88) una prova teorica. Correggere in 30 minuti e ripetere oralmente le dimostrazioni di BFS, Dijkstra e Kruskal.

Svolgere: (89) una ricorrenza divide et impera; (90) una DP su stringhe; (91) una DP su matrice; (92) un backtracking; (93) un rapporto di approssimazione. Per ogni traccia motivare perché il paradigma scelto è corretto.

Niente argomenti nuovi. Ripetere le complessità, le ipotesi e gli scheletri Python. Svolgere solo (94) un esercizio breve a scelta su grafi e (95) uno su DP. Controllare la checklist: idea, pseudocodice, prova, tempo, spazio, test.

Soluzioni-guida per l'autocorrezione

Albero: controllare $m=n-1$ e fare BFS/DFS; costo $O(n+m)$. Un ciclo non orientato si trova con DFS ignorando il padre. Ponte (u,v) : $low[v] > tin[u]$. Articolazione non radice: $low[v] \geq tin[u]$; radice: almeno due figli.

BFS salva distanza al primo ingresso e padre. Il diametro di un albero si ottiene con BFS da un nodo, poi BFS dal nodo più lontano. Kahn fallisce se estrae meno di n nodi. Nodi equidistanti soddisfano $ds[v] = dt[v]$.

Dijkstra rilassa con heap e richiede pesi non negativi. Bellman-Ford rilassa tutti gli archi $n-1$ volte e un miglioramento ulteriore segnala ciclo negativo. Floyd-Warshall usa intermedi progressivi e costa $O(n^3)$.

Kruskal sceglie archi ordinati che collegano componenti diverse; Union-Find evita cicli. La proprietà del taglio giustifica la scelta. Un matching M e un lower bound per Vertex Cover; gli estremi dei suoi archi danno un cover di dimensione $2|M|$, quindi fattore 2.

Esplicitare i sottoproblemi e il costo di combinazione. Ricerca binaria $O(\log n)$, massimo sottosegmento e coppia di punti più vicini $O(n \log n)$; Quickselect e $O(n)$ medio e $O(n^2)$ pessimo.

Una DP corretta ha stato sufficiente, basi e ordine aciclico. Piastrellamento $O(n)$; matrici $O(n^2)$; zaino $O(nC)$; LCS $O(pq)$; palindroma e Catalano $O(n^2)$. Per la soluzione, risalire dalla cella finale seguendo la scelta che ha creato il valore.

Il backtracking aggiunge una scelta, verifica vincoli, ricorre e annulla. N-regine usa colonne, $r-c$ e $r+c$. La

correttezza segue dal fatto che ogni configurazione ammissibile e costruita una volta e ogni ramo scartato viola un vincolo necessario.

Per tutte le soluzioni dettagliate, dopo aver confrontato l'idea, bisogna riscrivere il codice da zero e testare casi limite. Una soluzione letta non equivale a una soluzione saputa.

Registro giornaliero

Data: _____ Ore effettive: _____ Esercizi completati: _____

Errore principale: _____

Regola da ricordare domani: _____

Voto stimato della sessione (0-30): _____